

BIỂU DIỄN TRI THỨC VÀ TÌM KIẾM NÂNG CAO

Từ tìm kiếm đến chương trình

Giảng viên: Nguyễn Bích Vân IAI-UET-VNU

Mục tiêu

Tự tính được biên tìm kiếm, đường lời giải, g, h, f ; hiểu vai trò của hàm kinh nghiệm, tính chấp nhận được và nhất quán. Tự cài đặt, kiểm thử và giải thích năm thuật toán; phân biệt *chạy ra đáp án* với *hiểu vì sao mã trả về đáp án đó*.

Sản phẩm nộp

Bài 1–6: bản viết tay hoặc viết bút điện tử, gồm bảng vết chạy và lập luận; không thay bằng ảnh đầu ra chương trình. Bài 7–11: mã trong file `bai_lam.py`, kiểm thử do sinh viên tự viết, JSON/CSV kết quả, báo cáo nhận xét và hướng dẫn chạy. Có thể nộp hai phần trong cùng tệp ZIP.

Tham khảo mã mẫu để học, sau đó tự hoàn thiện bài được giao và ghi rõ các sửa đổi. Không import hàm tìm kiếm từ `*_demo.py` vào bài nộp. Các quy định tự làm bài của học phần vẫn áp dụng.

Lộ trình làm việc đề xuất

Làm tay trước để có dự đoán; chạy ví dụ mẫu để hiểu cấu trúc; viết từng hàm trong `bai_lam.py`; kiểm thử một hàm ngay khi hoàn thành; cuối cùng so sánh năm chiến lược và chuẩn bị giải thích trực tiếp trên lớp.

Quy ước dùng thông nhất trong đề, mã mẫu và bài làm

- Đồ thị và dữ liệu.** Hữu hạn, có hướng, chỉ dùng cạnh đã cho. Đầu là S , đích là G nếu không nói khác. Danh sách kề có thứ tự; không tự bổ sung cạnh ngược. Điểm giao nét vẽ không phải đỉnh. Trọng số dương; các số trong bài nhỏ và tính chính xác được.
- Greedy.** Chọn h nhỏ nhất trên *toàn bộ biên*. Biến thể của bộ này đánh dấu khi đưa vào biên, lưu cha lần đầu; không cập nhật theo g . g chỉ dùng tính chi phí đường trả về.
- UCS và A*.** Ưu tiên lần lượt g và $f = g + h$. Khi hòa ưu tiên, chọn tên đỉnh theo thứ tự từ điển. Chỉ cập nhật g và cha khi chi phí mới *nhỏ hơn hẳn*; bằng nhau giữ cha cũ. Các bản ghi chi phí cũ còn trong heap phải bị bỏ khi lấy ra.
- Dừng và đếm.** Kiểm tra đích *sau khi lấy bản ghi hợp lệ*, trước khi sinh hàng xóm. **order** gồm đích, không gồm bản ghi bị bỏ. **expanded** không tính đích, nhưng tính mỗi lần mở rộng lại. Đường lời giải dựng từ **parent**, không phải **order**.
- Hai phiên bản A*.** Mặc định cho phép mở lại: nếu có đường rẻ hơn vào đỉnh đã đóng, bỏ đỉnh khỏi CLOSED, cập nhật và đưa lại vào OPEN. Khi đề yêu cầu *không mở lại*, bỏ hẳn ứng viên vào đỉnh đã đóng, *không sửa cả cha lẫn g của đỉnh đó*. Bản này có bảo đảm tối ưu theo bài học khi h nhất quán.
- Bảng làm tay.** Viết phần tử sẽ lấy tiếp ở bên trái. Liệt kê *các bản ghi thực còn trong heap*, kể cả bản ghi cũ; đánh dấu bản ghi bị bỏ tại bước SKIP. Cách ghi các đỉnh trong biên: A*: u_g, f ; UCS: u_g ; Greedy: u_h . Với khởi tạo, biên chỉ có đầu.
- DFS.** Dùng stack chứa (đỉnh, cha dự kiến); đánh dấu/chốt cha ở lần pop hợp lệ đầu; đẩy hàng xóm theo thứ tự đảo. Bỏ bản ghi của đỉnh đã thăm. Trong Python, đỉnh stack ở cuối list; ảnh chụp giăng dạy được đảo để phần tử lấy tiếp ở trái.
- BFS.** Dùng deque, đánh dấu ngay khi enqueue, cha chỉ gán một lần. Không lấy trọng số làm tiêu chí ưu tiên.

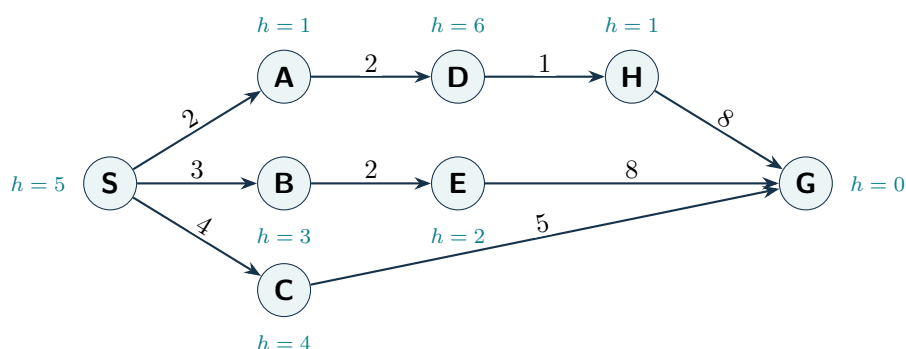
Phần I – Tính toán bằng tay

BÀI 1 | Greedy chọn trong toàn bộ biên

Tính tay – Trung bình

5 điểm

Một rô-bốt tìm đường trong mạng hành lang. Mỗi cạnh mang chi phí di chuyển; giá trị h là ước lượng chi phí còn lại để đến mục tiêu. Dùng Greedy theo quy ước chung.



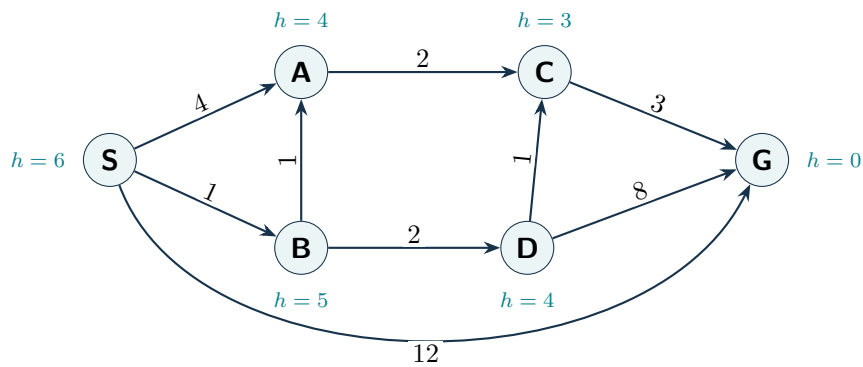
Đỉnh	Danh sách cạnh ra (v, c)	h
S	$(A, 2), (B, 3), (C, 4)$	5
A	$(D, 2)$	1
B	$(E, 2)$	3
C	$(G, 5)$	4
D	$(H, 1)$	6
E	$(G, 8)$	2
H	$(G, 8)$	1
G	$\emptyset$	0

- Lập bảng chạy thuật toán từ khởi tạo đến khi dừng, trong đó mô tả đỉnh lấy ra và biên sau xử lý, ghi h từng ứng viên.
- Sau khi xử lý A , vì sao thuật toán có thể chọn một đỉnh không phải hàng xóm của A ? Chỉ ra đỉnh đó bằng phép so sánh số.
- Ghi riêng **order**, đường trả về, số cạnh, tổng chi phí và số lần mở rộng. Liệt kê các đường hoàn chỉnh để xác định đường tối ưu thật sự.
- Từ dãy h của các đỉnh lấy ra, nhận xét phát biểu: “Greedy luôn lấy các đỉnh theo dãy h tăng dần”.

BÀI 2 | A*: cập nhật chi phí và thời điểm dừng

Tính tay – Trung bình

5 điểm



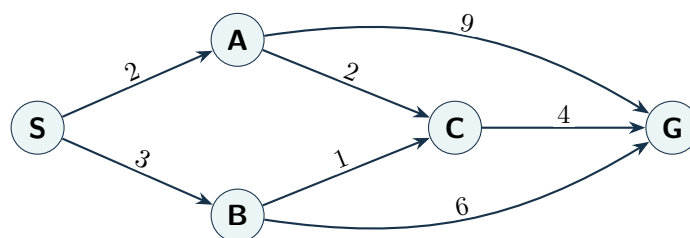
Đỉnh	Danh sách cạnh ra (v, c)	h
S	$(A, 4), (B, 1), (G, 12)$	6
A	$(C, 2)$	4
B	$(A, 1), (D, 2)$	5
C	$(G, 3)$	3
D	$(C, 1), (G, 8)$	4
G	$\emptyset$	0

- Chạy A*. Lập bảng OPEN sau mỗi bước; với mỗi ứng viên ghi (g, h, f) hoặc $u_{g,f}$. Ghi các lần đổi cha/chi phí.
- Có thể trả lời ngay khi sinh G lần đầu không? So sánh chi phí đường đó với kết quả khi dừng đúng quy tắc.
- Giải thích vì sao C được lấy trước D tại một bước đồng hạng; vì sao cha của C không đổi khi xét đường qua D.
- Xác định đường trả về, tổng chi phí, số lần mở rộng; chỉ ra một đường tối ưu khác. Kiểm tra tính nhất quán của h trên mọi cạnh.

BÀI 3 | Kiểm tra ba bảng hàm kinh nghiệm

Tính tay – Trung bình

5 điểm



Đỉnh	Danh sách cạnh ra (v, c)
S	$(A, 2), (B, 3)$
A	$(C, 2), (G, 9)$
B	$(C, 1), (G, 6)$
C	$(G, 4)$
G	$\emptyset$

Đỉnh	S	A	B	C	G
h_1	6	4	3	2	0
h_2	8	6	5	1	0
h_3	8	8	5	4	0

- Tính $h^*(u)$ cho mọi đỉnh bằng cách đi ngược từ G ; ghi phép lấy min ở đỉnh có nhiều đường.
- Với mỗi bảng h_i , kết luận có chấp nhận được không. Nếu không, chỉ rõ đỉnh và bất đẳng thức vi phạm.
- Kiểm tra tính nhất quán; liệt kê *tất cả* các cạnh vi phạm nếu có.
- Chạy A* với h_3 . Kết quả trên dữ liệu này có tối ưu không? Kết quả một ví dụ có đủ để suy ra bảo đảm tối ưu cho mọi bài toán không?

BÀI 4 | Trò đẩy chữ: từ ước lượng đến các bước A*

Tính tay – Trung bình

5 điểm

Ô trống ký hiệu 0 khi mã hóa. Mỗi bước đổi chỗ ô trống với một ô số kề ngang/dọc, chi phí 1. Trạng thái đích giữ cách đặt ô trống góc trên trái như ví dụ trong bài giảng.

1	2	5
3		4
6	7	8

Bắt đầu

	1	2
3	4	5
6	7	8

Mục tiêu

Đặt h_P = số ô số sai vị trí; h_M = tổng khoảng cách Manhattan của 8 ô số tới vị trí đích. **Không tính ô trống vào hai hàm.**

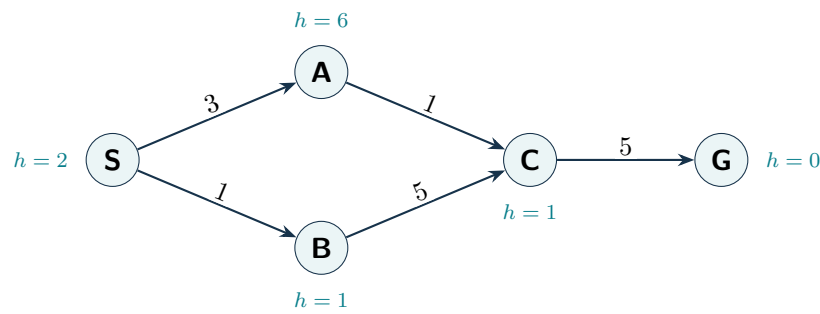
- Tính $h_P(S)$ và $h_M(S)$, lập bảng đóng góp Manhattan của từng ô số.
- Liệt kê bốn trạng thái sau một bước theo thứ tự di chuyển *ô trống*: Phải, Xuống, Trái, Lên. Với mỗi trạng thái tính $g, h_P, h_M, f = g + h_M$.
- Chạy A* với h_M đến đích; không giữ đường quay lại trạng thái đã biết với g lớn hơn hoặc bằng. Khi bằng f , so chuỗi 9 chữ số đọc theo hàng, kể cả 0. Ghi dãy trạng thái lấy ra, chi phí và các phép di chuyển ô trống.
- Chứng minh đường tìm được là tối ưu bằng một cận dưới cụ thể. Nếu cộng khoảng cách của cả ô trống vào h_M , trên chính dữ liệu này có còn chấp nhận được không?

Manhattan của ô số k tại (r, c) , đích (r_k, c_k) là $|r - r_k| + |c - c_k|$. Ví dụ trạng thái đầu mã hóa thành 125304678. Hãy phân biệt chuyển động ô trống với chuyển động của ô số.

BÀI 5 | A* trên đồ thị: cái giá của việc không mở lại

Tính tay – Khó

10 điểm



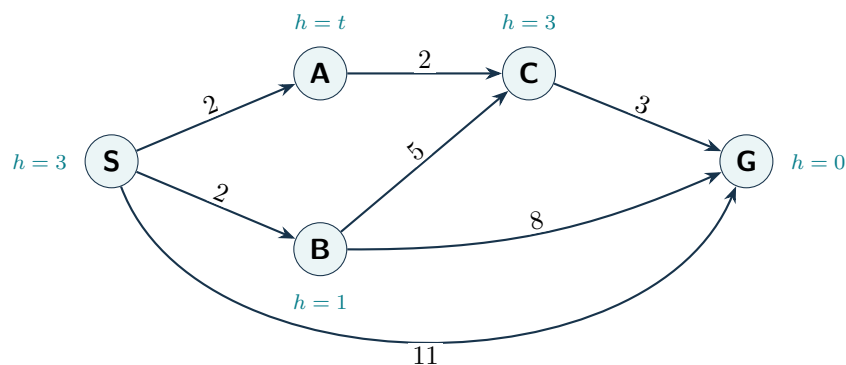
Đỉnh	Danh sách cạnh ra (v, c)	h
S	$(A, 3), (B, 1)$	2
A	$(C, 1)$	6
B	$(C, 5)$	1
C	$(G, 5)$	1
G	$\emptyset$	0

- Tính h^* và kiểm tra h chấp nhận được. Xác định toàn bộ cạnh vi phạm tính nhất quán.
- Chạy A* **không mở lại**: khi xét cạnh đi vào đỉnh đã đóng thì bỏ ứng viên trước khi cập nhật g /cha. Lập bảng OPEN, CLOSED và đường trả về. Vì sao kết quả có thể sai tối ưu?
- Chạy lại A* **có mở lại**. Chỉ rõ thời điểm cải thiện $g(C)$, cách thay đổi CLOSED/cha, dãy f của các lần lấy ra hợp lệ và kết quả cuối cùng.
- So sánh số lần mở rộng với số trạng thái khác nhau được mở rộng. Có thể khẳng định A* luôn mở rộng mỗi trạng thái tối đa một lần khi h chỉ chấp nhận được không?
- Giữ các giá trị h khác, chỉ sửa $h(A)$. Tìm giá trị lớn nhất của $h(A)$ để toàn bộ bảng nhất quán. Dùng bảng đã sửa để giải thích vì sao không cần mở lại trong ví dụ này.

BÀI 6 | Tham số heuristic: bảo đảm lý thuyết và kết quả thực tế

Tính tay – Khó

10 điểm



Đỉnh	Danh sách cạnh ra (v, c)
S	$(A, 2), (B, 2), (G, 11)$
A	$(C, 2)$
B	$(C, 5), (G, 8)$
C	$(G, 3)$
G	$\emptyset$

Cho $h(S) = 3$, $h(A) = t$, $h(B) = 1$, $h(C) = 3$, $h(G) = 0$, với $t \geq 0$. A* dùng bản có mở lại. Khi hòa f , ưu tiên tên đỉnh; do đó A trước C trước G .

- Tính h^* ; tìm tất cả t để h chấp nhận được, và tất cả t để h nhất quán. Viết các bất đẳng thức dẫn tới hai miền này.
- Lập bảng A* riêng cho $t = 0, 5, 8, 9$. Ghi thứ tự lấy ra, đường và chi phí. Chú ý các giá trị f có thể giảm sau khi sinh đỉnh mới khi h không nhất quán.
- Suy ra công thức từng khoảng cho chi phí đường A* trả về, với *mọi* $t \geq 0$. Giải thích chính xác hai biên $t = 1$ và $t = 8$, không chỉ suy đoán từ bốn lần chạy.
- Xác định chi phí Greedy cho mọi $t \geq 0$. So sánh ba miền: h nhất quán, h chấp nhận được và A* tình cờ trả kết quả tối ưu trên đồ thị này.

Phần II – Thực hành Python

Mở thư mục gốc `Greedy_AStar_11Bai` bằng VS Code. Tất cả tệp chạy chính đều ở thư mục gốc, không cần gõ `cd python`. Trước hết đọc `HuongDan_CaiDat_Python.pdf`; chạy mẫu bằng `python run_demo.py -algo astar -step`.

Hợp đồng hàm và dữ liệu

Dùng Python 3.10+ và thư viện chuẩn. Đồ thị có khóa là chuỗi khác rỗng, giá trị là list các cặp (v, w) ; không có cạnh song song cùng hướng. Mọi đỉnh kề/đầu/đích đều phải có khóa. Trọng số là int/float hữu hạn dương (không nhận bool). Bảng h có đúng tập đỉnh, hữu hạn không âm, $h(goal) = 0$. Dữ liệu sai phải báo `ValueError`; không sửa dữ liệu đầu vào. Được dùng bộ kiểm tra và các hàm phụ trợ trong `common.py`.

```
def dfs(graph, start, goal, step=False) -> dict: ...
def bfs(graph, start, goal, step=False) -> dict: ...
def ucs(graph, start, goal, step=False) -> dict: ...
def greedy(graph, start, goal, h, step=False) -> dict: ...
def astar(graph, start, goal, h, step=False, reopen=True) -> dict: ...
```

Trường kết quả	Ý nghĩa
<code>path, cost</code>	Danh sách đỉnh trên đường; thất bại: <code>None</code> và <code>math.inf</code> . Đầu trùng đích: đường một đỉnh, giá 0.
<code>order</code>	Thứ tự lấy ra hợp lệ; có thể lặp đỉnh ở A* mở lại.
<code>expanded, skipped</code>	Số lần mở rộng (không tính đích), số bản ghi bị bỏ.
<code>reopened</code>	Số lần một đỉnh bị lấy khỏi CLOSED do có g tốt hơn; các thuật toán khác trả 0.
<code>max_frontier</code>	Số bản ghi lớn nhất trong cấu trúc thật, kể cả bản ghi cũ.
<code>trace</code>	List sự kiện: <code>event, node, before, after, note</code> . Lưu snapshot độc lập trước/sau.

Cách tự kiểm tra và nộp

Hoàn thiện một hàm trong `bai_lam.py`, chạy bài tương ứng bằng `python chay_bai_lam.py -bai 7` (đổi số bài). Tệp `test_cong_khai.py` chỉ kiểm tra tình huống nhỏ; sinh viên phải viết thêm `test_bai_lam.py`. Kết quả chính được lưu tự động trong `output/` dưới dạng JSON và CSV.

Trong báo cáo ghi: mô hình, 3–5 bước dự đoán trước khi chạy, kết quả, một kiểm thử biên, một thay đổi dữ liệu, một lỗi đã sửa hoặc điểm hiểu rõ hơn. Gói `MSSV_HoTen_Search.zip` gồm mã, helper/dữ liệu cần chạy, kiểm thử, output, báo cáo và README.

BÀI 7 | DFS: chu trình, cha và thứ tự nhánh

Python – Cài đặt bằng ngăn xếp

12 điểm

Đỉnh	Danh sách cạnh ra (v, c)
S	$(A, 1), (B, 1)$
A	$(C, 1), (D, 1)$
B	$(D, 1), (G, 1)$
C	$(A, 1), (E, 1)$
D	$(G, 1)$
E	$(B, 1)$
G	$\emptyset$
X	$\emptyset$

- Viết **dfs** bằng ngăn xếp tường minh, không đệ quy. Mỗi bản ghi chứa đỉnh và cha dự kiến; đánh dấu/chốt cha khi pop hợp lệ, đẩy hàng xóm đảo thứ tự. Dùng bảng cha dựng đường.
- Chạy $S \rightarrow G$ và $S \rightarrow X$; in đường, số cạnh, **order**, **expanded**, **skipped**, **max_frontier**. Chọn một bước có đỉnh đang chờ xuất hiện nhiều lần để giải thích.
- Đảo riêng thứ tự hai cạnh ra của S trên một bản sao. Dự đoán đường mới rồi chạy. Giải thích tại sao DFS không bảo đảm ít cạnh nhất.
- Tự kiểm thử đầu trùng đích, khuyên $S \rightarrow S$, đỉnh đích thiếu khóa, không sửa đầu vào. Với $S \rightarrow X$, kiểm tra rằng chương trình kết thúc dù có chu trình.

Minh chứng phải nộp

Bảng 3 bước đầu, kết quả hai đích, kết quả đảo thứ tự, ít nhất 4 kiểm thử và giải thích 100–150 từ về thời điểm gán cha. Dùng **-step** để kiểm tra snapshot; không nhầm đỉnh stack ở cuối list với cách in phần tử lấy tiếp ở trái.

BÀI 8 | BFS: tìm đường ít bước trong mê cung

Python – Mô hình hóa và hàng đợi

12 điểm

```
maze = ["S..#..", ".#.#G.", "...#..", "....."]
```

	0	1	2	3	4	5
0	S					
1					G	
2						
3						

- Viết **bfs** bằng **deque**; đánh dấu khi enqueue, lưu cha lần đầu. Được dùng **maze_graph** và **draw_maze** trong **common.py**; giải thích vì sao một ô đi được tương ứng một đỉnh.
- Chỉ đi ngang/dọc, thứ tự Phải–Xuống–Trái–Lên, phí mỗi bước 1. Chạy mê cung, xuất đường dạng tên ô **"r,c"**, số bước, **order**, số lần mở rộng và hình ký tự có dấu *****.
- Chặn ô (3,3) bằng cách thay hàng cuối thành **"...#.."**, tức chuỗi chính xác trong mã dưới đây. Dự đoán còn đường tới G không; chạy và giải thích vùng còn đến được.
- Kiểm thử đầu trùng đích ở hàm BFS; bảng mê cung rỗng, không chữ nhật, ký tự lạ hoặc thiếu/nhân đôi S/G phải bị phát hiện. Không xóa S/G hoặc tường khi vẽ đường.

```
blocked = ["S..#..", ".#.#G.", "...#..", "...#.."]
```

Minh chứng phải nộp

Hình ký tự, 4 trạng thái hàng đợi đầu, giải thích vì sao số ô được xét không bằng độ dài đường, ít nhất 4 kiểm thử. Trong chế độ xem, nhấn Enter ở *Terminal*, không ở cửa sổ Output.

BÀI 9 | UCS: cập nhật đúng và bỏ bản ghi cũ

Python – Hàng đợi ưu tiên theo chi phí

12 điểm

Đỉnh	Danh sách cạnh ra (v, c)
S	$(A, 8), (B, 2), (G, 20)$
A	$(C, 3), (G, 9)$
B	$(A, 1), (D, 2)$
C	$(G, 4)$
D	$(C, 1), (G, 10)$
G	$\emptyset$
X	$\emptyset$

- Viết `ucs` với heap (g, node) ; dùng `best_g`, bảng cha, cập nhật khi nhỏ hơn hẳn và bỏ bản ghi cũ trước khi kiểm tra đích.
- Chạy $S \rightarrow G$; lưu trace đầy đủ, đánh dấu lần cải thiện A , C và những bản ghi lỗi thời. So sánh đường UCS với đường BFS trên cùng đồ thị theo *chi phí gốc*.
- Đổi mọi trọng số về 1 trên bản sao; so sánh chi phí và số cạnh với BFS. Không đòi hai thuật toán có cùng `order` nếu quy tắc đồng hạng khác nhau.
- Tự tạo hai kiểm thử khác nhau: (i) hai đỉnh có cùng g trong heap; (ii) một đỉnh có hai đường cùng g và cha cũ phải được giữ. Kiểm thử thất bại, đầu trùng đích và trọng số sai (0, âm, inf, NaN).

Minh chứng phải nộp

Chỉ rõ dòng cập nhật cả giá và cha; giải thích bằng dữ liệu vì sao không được dừng ngay khi sinh G , và vì sao không thể khóa chi phí ngay lần phát hiện đầu.

BÀI 10 | Greedy: thay đổi ước lượng, thay đổi quyết định

Python – Ưu tiên h trên toàn bộ biên

12 điểm

Đỉnh	Danh sách cạnh ra (v, c)	h
S	$(A, 1), (B, 3), (C, 2)$	4
A	$(D, 2)$	1
B	$(E, 1)$	2
C	$(G, 4)$	3
D	$(G, 9)$	7
E	$(G, 8)$	1
G	$\emptyset$	0
X	$\emptyset$	0

Đỉnh	S	A	B	C	D	E	G	X
h_2	4	6	5	1	7	4	0	0

- Viết **greedy**: heap ($h, node$), đánh dấu khi thêm, giữ cha lần đầu. Trọng số không tham gia khóa ưu tiên nhưng phải được dùng khi tính giá đường trả về.
- Chạy với h ở bảng đầu. Chỉ ra một bước thuật toán rồi nhánh vừa xét để chọn một ứng viên đã có trong biên. So sánh với đường tối ưu bằng UCS.
- Chạy lại với h_2 và với $h_0 \equiv 0$. Dự đoán trước đường, giá, số lần mở rộng; giải thích khác biệt. Tự tạo một đồ thị nhỏ cho thấy Greedy với h_0 không nhất thiết có cùng **order** như BFS.
- Kiểm thử hòa h , chu trình, đích không tới được và bảng h sai (thiếu đỉnh, âm hoặc $h(G) \neq 0$). Không viết thuật toán chỉ chọn hàng xóm có h nhỏ nhất của đỉnh hiện tại.

BÀI 11 | A*: đường rẻ, heuristic và mở lại trạng thái

Python – Cài đặt, đối chiếu và giải thích

12 điểm

```
rows = ["SwG", "....", ".##.", "...."]
```

	0	1	2	3
0	S	w	w	G
1				
2				
3				

Phí đi vào ô **w** là k , vào ô khác đi được là 1; $k \in \{1, 5, 8\}$. Không thu phí ở ô xuất phát. Dùng h Manhattan tới G, không tính tường/nước vào ước lượng. Tọa độ và thứ tự láng giềng như Bài 8.

- Viết **astar** với ($f, node, g$), $f = g + h$, **best_g**, bảng cha và kiểm tra bản ghi cũ. Hỗ trợ **reopen=True/False** đúng quy ước; giải thích khi nào bản không mở lại có bảo đảm theo bài học.
- Với $k = 5$, chạy BFS, UCS, Greedy, A*. Lập bảng đường, số cạnh, chi phí, số lần mở rộng, đỉnh biên lớn nhất. Không kết luận “luôn nhanh hơn” chỉ từ một ví dụ.
- Chạy lại với $k = 1, 8$; chứng minh Manhattan nhất quán với mô hình phí này. Cho $h = 0$ và kiểm tra A* có cùng đường, giá, **order** như UCS khi dùng cùng quy tắc hòa.
- Chạy đồ thị Bài 5 với bảng h ở đó, lần lượt không/có mở lại. Đối chiếu lời làm tay, chụp snapshot lần cải thiện C , kiểm thử kết quả bằng chi phí liệt kê đường đơn hoặc UCS.

Minh chứng phải nộp

Mã hoàn chỉnh; bảng so sánh; kiểm thử thất bại, đầu trùng đích, đồng hạng và mở lại; đoạn giải thích 150–200 từ phân biệt g , h , f , **discovered** và **closed**. Được dùng bộ chuyển mê cung đã cho; không dùng hàm A* có sẵn.